Data Structures Lab-07

Heap Data Structures

Duration	150 Minutes (2 Hours 30 Minutes)
Programming Language	Java
Lab	07
Topics	Max Heap, Min Heap, Heap Sort

1. Learning Outcomes

- Explain the structure and properties of a binary heap.
- Differentiate between Min Heap and Max Heap.
- Represent a heap efficiently using an array.
- Implement insertion using heapify-up.
- Implement root deletion using heapify-down.
- Implement Heap Sort and analyze its complexity.
- Apply heaps to practical problems such as Priority Queues.

3. Lab Schedule — 150 Minutes

Activity	Time
Heap concepts and demonstration	15 min
Min Heap vs. Max Heap + array formulas	15 min
Task 1 — Implement Max Heap	30 min
Task 2 — Delete Root + Heapify Down	25 min
Task 3 — Heap Sort	30 min
Task 4 — Min Heap Challenge	10 min
Testing, debugging & viva	25 min
TOTAL	150 min

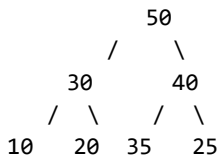
4. Concept Brief

A heap is a complete binary tree that satisfies an ordering property. In a Max Heap, every parent is greater than or equal to its children. In a Min Heap, every parent is smaller than or equal to its children.

For an array-based heap using zero-based indexing:

Relationship	Formula
Parent	(i - 1) / 2
Left Child	2 × i + 1
Right Child	2 × i + 2

MAX HEAP



5. Task 1 — Implement a Max Heap

Create a Java class named MaxHeap that stores integer values in an array and maintains the Max Heap property after every insertion.

Required Operations

- insert(int value)
- display()
- parent(int index)
- leftChild(int index)
- rightChild(int index)

6. Task 2 — Implement Heap Sort

Implement Heap Sort using a Max Heap. Build the heap, repeatedly move the maximum element to the end of the unsorted region, reduce the heap size, and heapify.

Required Methods

- buildMaxHeap(int[] arr)
- heapify(int[] arr, int n, int i)
- heapSort(int[] arr)
- display(int[] arr)

Example:

```
[40, 10, 30, 50, 20, 60]  →  [10, 20, 30, 40, 50, 60]
```

8. Task 3 — Min Heap Challenge

Modify the Max Heap logic to create a MinHeap class. Implement insertion, deleteMin(), heapifyDown(), and display().

9. Complexity Analysis

Complete the table before submitting your lab.

Operation	Time Complexity
Access root	O(1)
Insert	O(log n)
Delete root	O(log n)
Heapify	O(log n)
Build Heap	O(n)
Heap Sort	O(n log n)
Search arbitrary element	O(n)

11. Viva Questions

1. What is a Heap and what makes it a complete binary tree?
2. What is the difference between a Min Heap and a Max Heap?
3. Why is an array suitable for storing a heap?
4. What are the formulas for parent, left child, and right child?
5. What is heapify-up?
6. What is heapify-down?
7. What happens when the root of a heap is deleted?
8. What is the time complexity of insertion and deletion?
9. Why is Heap Sort O(n log n)?
10. How can a heap be used to implement a Priority Queue?

12. Submission Checklist

Item	Completed
HeapSort.java	☐

END OF LAB • GOOD LUCK!